Wenqing Deng

+1 858-370-2174 | w2deng@ucsd.edu | La Jolla, CA

SUMMARY

Seeking a full-time Software Engineer position starting July 2027.
Rising senior in Mathematics-Computer Science with software engineering internship experience and diverse projects across full-stack development, backend systems, AI/RAG applications, and Android development.

EDUCATION

University of California, San Diego **San Diego, CA**
B.S. in Mathematics-Computer Science Sep 2023 – Jun 2027
Honor: USACO Gold
Coursework: Data Structures, Systems Programming, Machine Learning, Recommender Systems, OO Design, Graph Theory, Probability, Cryptography

TECHNICAL SKILLS

Programming Languages: Java, Kotlin, Python, C, Go, JavaScript, TypeScript, SQL, MATLAB
Frameworks: React, Next.js, Spring Boot, JavaFX, Jetpack Compose, Express, LangChain, MUI, Ant Design
Data & Tools: PostgreSQL, SQLite, Room, Elasticsearch, Google Cloud Storage, REST, Retrofit, Ktor, MCP, Git, Docker, Maven, Gradle, JUnit

WORK EXPERIENCE

Hirebeat **Jersey City, NJ**
Software Development Engineer Intern Jul 2025 – Sep 2025

- Refactored Google reCAPTCHA v3 into an app-level React provider in Next.js, centralizing token execution across the registration flow and removing page-level script handling.
- Fixed registration confirmation to use the submitted email value and added a cancel path; built reusable no-sidebar and minimal-app-bar layouts for the account activation screen.
- Updated notification handling with functional React state setters to prevent stale-state overwrites, and constrained long notification content with dedicated scroll regions.

PROJECTS

Agent AI PDF Assistant

- Built a full-stack assistant that uploads PDFs, splits and embeds document text, retrieves relevant chunks from an in-memory vector store, and produces source grounded answers through a RAG pipeline.
- Implemented PDFLoader, recursive chunking, OpenAI embeddings, similarity retrieval, and prompt assembly in an Express service; cached the vector index to avoid re-embedding the same document.
- Integrated a local MCP server with SerpAPI and ran RAG/web requests concurrently, returning partial results when one provider fails; added browser voice interaction through text to speech and speech to text.

Online Food Ordering Platform

- Developed a full-stack ordering workflow with restaurant/menu discovery, user registration and sign-in, session-based authorization, cart quantity management, checkout, and centralized API error handling.
- Designed Spring Data JDBC entities, repositories, transactional cart services, and REST controllers backed by PostgreSQL; connected a React/Ant Design client with credentialed requests and live cart state.

Social AI Media Platform

- Implemented a Go REST API with JWT-protected upload, search, and delete routes plus public sign-up/sign-in endpoints; stored searchable user and post metadata in Elasticsearch.
- Integrated Google Cloud Storage and a React/Material UI client for authenticated publishing, search, gallery browsing, and AI image generation.

Recommendation System Modeling & Evaluation Pipeline

- Built a multi-task pipeline for rating regression and anime-type multiclass classification using cleaned user ratings and engineered metadata features.
- Compared baseline, Random Forest, Gradient Boosting, and XGBoost models using RMSE/R-squared, macro F1, residuals, and confusion matrices.

Spotify-Style Android Music App

- Built home, playlist, favorites, and persistent player-bar experiences with Jetpack Compose, MVVM, StateFlow, Hilt dependency injection, and Android Navigation.
- Created repository/ViewModel data flows for Retrofit feed and playlist endpoints served by a Ktor backend, with loading/error state exposed to Compose through StateFlow.
- Implemented Room-backed favorite synchronization and an Activity-scoped ExoPlayer controller so play/pause, progress, and current-track state persist across screens.

Student Clock

- Built a JavaFX study planner that tracks courses, assignments, logged sessions, completion status, and progress against estimated effort through dashboard and timeline views.
- Worked with Presenter/Observer and Repository layers plus Builder/Factory patterns to separate JavaFX views, domain calculations, and SQLite persistence while keeping screens synchronized.
- Expanded JUnit user-story coverage for progress calculations, behind-schedule sorting, mock-time behavior, restart persistence, and mark-as-done workflows.